项目 2 开发环境入门

1、 预备知识

利用宇树科技为 Go2 机器人量身定制的软件开发工具包 (SDK)，我们得以构建一个稳定且功能丰富的开发环境。该 SDK 提供了完善的开发接口和丰富的应用示例，极大地便利了用户针对特定应用场景，开发和定制专属的应用程序，从而充分发挥 Go2 机器人平台的潜力。

1.1、 Go2 SDK 的作用

1.1.1、 编程便利性

Go2 SDK 提供了一套预先构建的代码库，使得开发者无需从头开始编写复杂的代码，就能够快速实现 Go2 机器人的控制和编程 。

1.1.2、 硬件抽象

Go2 SDK 通常封装了底层硬件的细节，使得开发者可以更加专注于应用层面的开发，而不必关心硬件的具体实现 。

1.1.3、 功能扩展

通过 Go2 SDK，开发者可以为 Go2 机器人添加新的功能，如视觉识别、人体追踪、自主导航等，从而扩展机器人的应用范围 。

1.1.4、 标准化接口

Go2 SDK 提供了标准化的接口，在宇树各种型号的机器人之间保持一定的通用性 。

2、 Go2 机器人开发架构

2.1、 控制原理

Go2 机器人为了满足不同场景和用户的需求，通常提供多种控制方式。这些控制方式包括但不限于遥控器控制、APP 控制以及 SDK 控制。

在 Go2 机器人的应用开发中，可以同时使用遥控器、APP 和 SDK，以适应不同的教学和实验需求。遥控器控制、APP 控制和 SDK 控制各有优势和适用场景。在设计 Go2 机器人时，根据应用场景和用户需求，选择合适的控制方式或将它们结合起来，可以提供更好的用户体验和系统性能。

2.2、 系统架构

Go2 机器人整体系统架构如下图，主要分为四个功能模块。

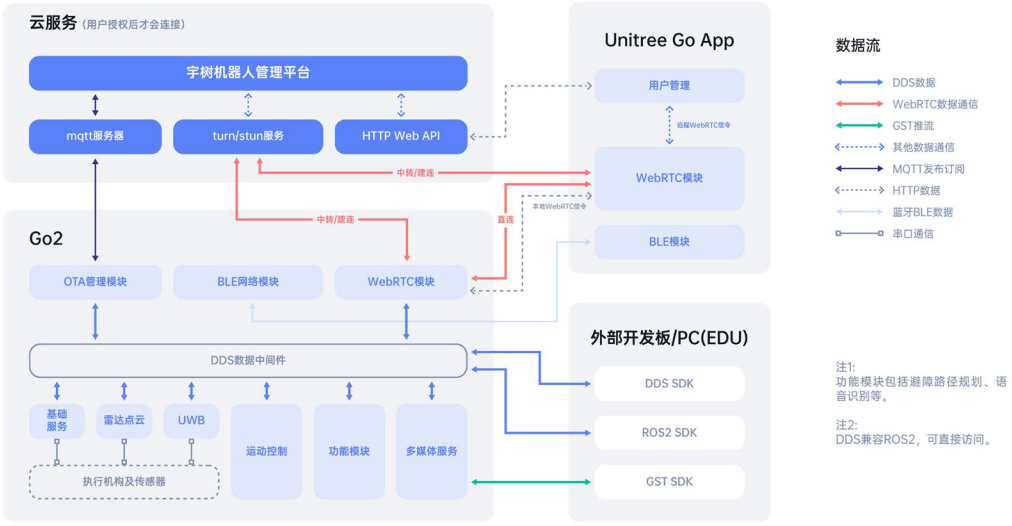


图 2-1 系统架构

2.2.1、 云服务

2.2.1.1、 数据收集

收集 Go2 运行数据（不涉及隐私），进行故障检测和统计。

2.2.1.2、 远程操作

帮助用户实现远程操作，主要通过 WebRTC 实现。图像和控制流量可以点对点或通过 turn server 转发，服务器不会收集或分析这些流量。

2.2.1.3、 系统迭代

进行系统升级迭代。

2.2.1.4、 通信功能

mqtt 用来建立与每条机器狗的物联网通信，负责监控故障、系统升级、传递 WebRTC 信令。

http 服务连接 App 和 Web 前端，建立用户与狗的绑定关系。

turn/stun 服务器用来帮助 WebRTC 点对点连接，并在无法实现点对点连接时提供服务器数据转发。

2.2.2、 Go2 机器人

OTA 模块通过 mqtt 与云服务器通信，负责上传故障信息、系统升级、并转发 WebRTC 信令。

WebRTC 模块实现与 App 的主要数据管道，包括音视频流、雷达点云、运动状态及控制指令。

蓝牙(BLE)部分用来和 App 建立联系，主要用于配置网络和安全验证。

各个功能模块之间的通信主要采用 DDS 实现，DDS IDL 兼容 ROS2 （需要选择适配的 RMW），EDU 版本可以通过 DDS 或 ROS2 调用接口。

电机、雷达、UWB 等传感器数据通过串口收集后再转发到 DDS 中间层。

EDU 及以上版本的多媒体模块面向二次开发，支持 GST 推流。

2.2.3、 App 程序

用户管理模块，通过 HTTP Web API 连接宇树管理平台。负责绑定机器狗、 WebRTC 建立连接等功能。

Go2 的蓝牙模块，用来配置网络。

WebRTC 模块，主要数据流量都是通过 WebRTC 实现，包括图传、点云、运动状态及控制指令下发。

2.3、 通信机制

Go2 机器人中主要采用了三种消息通信机制，分别是 DDS 协议、ROS2 协议和 UDP 协议，三种协议的简介如下：

DDS 协议： DDS (Data Distribution Service) 是一种高性能、可扩展的中间件协议，用于分布式实时系统中的数据通信。DDS 以数据为中心，提供了丰富的 QoS（Quality of Service）策略，包括可靠性、持久性、传输优先级等，允许开发者根据应用需求进行细粒度的控制。DDS 是无中心化的，支持点对点通信，可以减少中心节点故障的风险，并且支持数据的自动发现和分发。

ROS2 协议： ROS2（Robot Operating System 2）是 ROS 的第二代版本，它在通信方面采用了 DDS 作为底层通信机制。ROS2

协议以服务和动作为核心，提供了节点间的消息传递、服务请求和动作调用等通信机制。ROS2 的设计目标是支持大规模、高性能和安全性的机器人应用，同时保持跨平台的特性。ROS2 通过 RMW（ROS Middleware）抽象层实现对不同 DDS 实现的兼容性，使得开发者可以根据项目需求选择不同的 DDS 实现。

UDP 协议： UDP（User Datagram Protocol）是一种简单的无连接传输层协议，它在网络中提供快速的数据传输服务，但不保证数据的可靠性和顺序。UDP 适用于对实时性要求高但可以容忍一定丢包率的应用，如视频流、在线游戏和 VoIP 等。UDP 的头部开销小，因此传输速度快，但它不提供 TCP 协议中的流量控制、拥塞控制、错误恢复和数据顺序保证等功能。

DDS 协议和 ROS2 协议都是基于发布/订阅模型，而 UDP 协议是面向无连接的传输层协议。DDS 协议和 ROS2 协议常用于复杂的机器人和自动化系统，UDP 协议适用于实时音视频通信。Go2 机器人中采用 DDS 消息中间件与 DDS 应用、ROS 应用通信，采用 GST 基于 UDP 协议进行视频推流。整体消息机制及处理见下图：

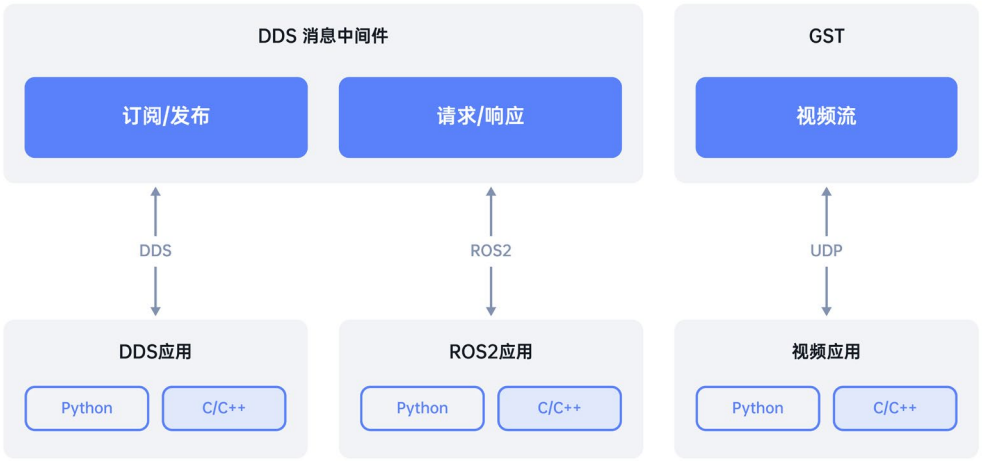


图 2-2 消息机制

Go2 采用 DDS 作为消息中间件，主要的数据交互采用两种模式： 订阅/发布和请求/响应。

订阅/发布： 接收方订阅某个消息，发送方根据订阅列表向接收方发送消息，主要用于中高频或持续的数据交互。

请求/响应： 问答模式，通过请求实现数据获取或操作。用于低频或功能切换时的数据交互。

请求响应接口的调用方式：API 调用，函数式调用

API 调用： 类似 restapi，请求发送时填入请求内容和请求话题, 在对应的响应话题接受回复，回复采用广播模式，根据 UUID 确定请求和响应的对应关系。

函数式调用： API 调用模式的语法糖，将 API 调用封装成函数

调用，方便用户使用。

3、 C++开发环境构建

以下安装流程适用于用户自选的开发主机或 Go2 机器人上的拓展坞系统，整体开发环境基于 Ubuntu 20.04 系统，工作目录（working directory）的根目录为用户主目录。

3.1、 unitree_sdk2 下载及安装

下载 unitree_sdk2 压缩包并解压至用户主目录。打开一个终端，并依次执行下列命令以安装 unitree_sdk2:

```
cd ~/unitree_sdk2/
mkdir build
cd build
cmake ..
sudo make install
```

指令 3.1 SDK 安装到根目录

或者安装到指定目录下::

```
cd ~/unitree_sdk2/
mkdir build
```

```
cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/opt/unitree_robotics
sudo make install
```

指令 3.2 SDK 安装到指定目录

安装说明：在上述命令中，CMAKE_INSTALL_PREFIX 后面用于指定将 unitree_sdk2 的安装至 /opt/unitree_robotics 目录下，如果您想将 unitree_sdk2 安装在其他目录，修改此路径即可。

3.2、 例程编译

在根目录打开一个终端，并依次执行下列命令以编译例程：

```
cd ~/unitree_sdk2
mkdir build
cd build
cmake ..
make
```

指令 3.3 例程编译

运行上文中的 make 命令后，若进度进行到 100%且没有报错，则意味着编译成功。

```
[  5%] Built target test_jsonize
[ 10%] Built target wireless
[ 15%] Built target humanoid
[ 23%] Built target HelloworldPublisher
[ 31%] Built target test_subscriber
[ 39%] Built target HelloworldSubscriber
[ 44%] Built target test_server_no_lease
[ 50%] Built target low_level
[ 57%] Built target test_publisher
[ 65%] Built target high_follow_csv
[ 71%] Built target high_level
[ 76%] Built target test_server
[ 81%] Built target test_client
[ 86%] Built target test_client_no_lease
[ 94%] Built target high_follow_sin
[100%] Built target go2_sub
```

图 3-1 例程编译

若成功执行 make 命令，生成的例程/二进制文件会在 build/bin 目录下。

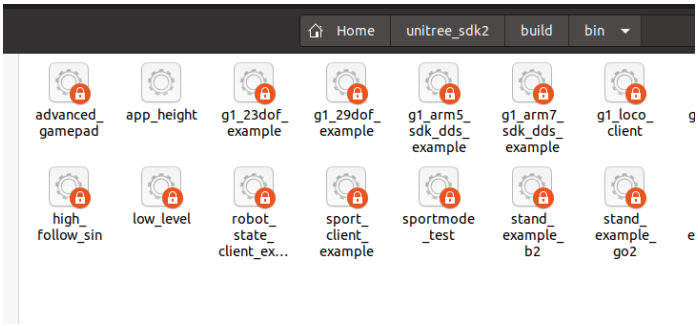


图 3-2 例程生成

3.3、 设置用户电脑的 IP 地址

Go2 机器人的内部硬件架构，由一块主控板、一块运动控制主板、一块或多块感知主板组成，内部一个交换机将各个板卡连起来成为一个局域网，局域网连接图如下：

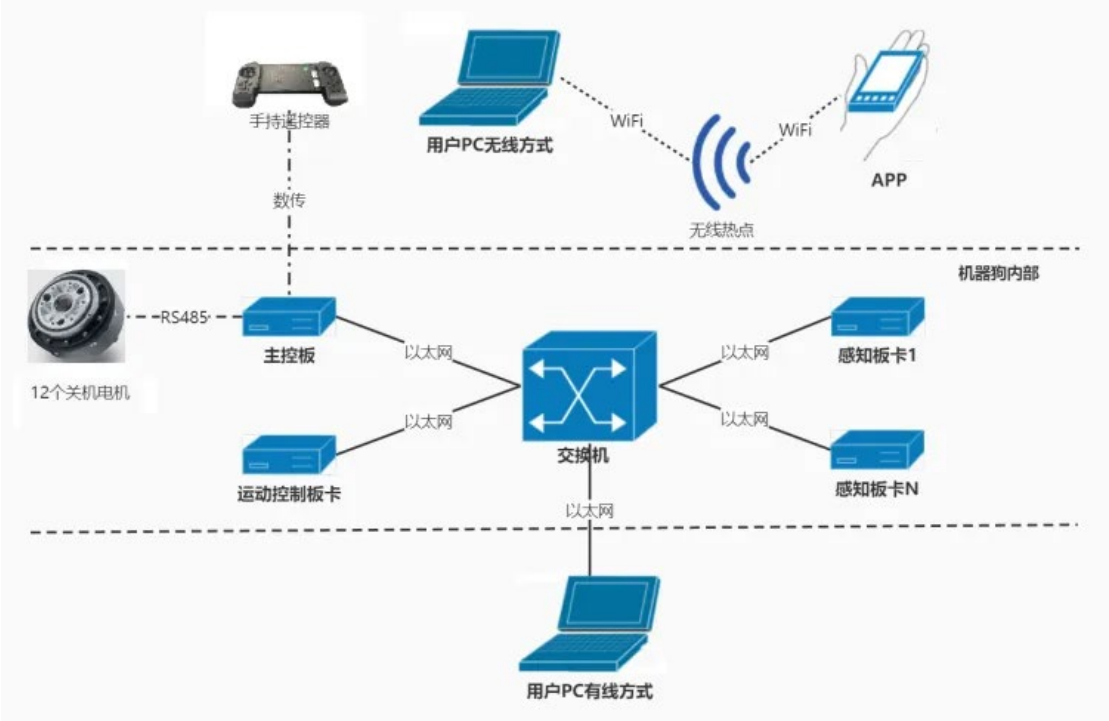


图 3-3 Go2 机器人网络架构

使用自己的电脑（Ubuntu 系统）以太网口连接 Go2 的以太网口，来建立用户电脑和 Go2 之间的通讯。

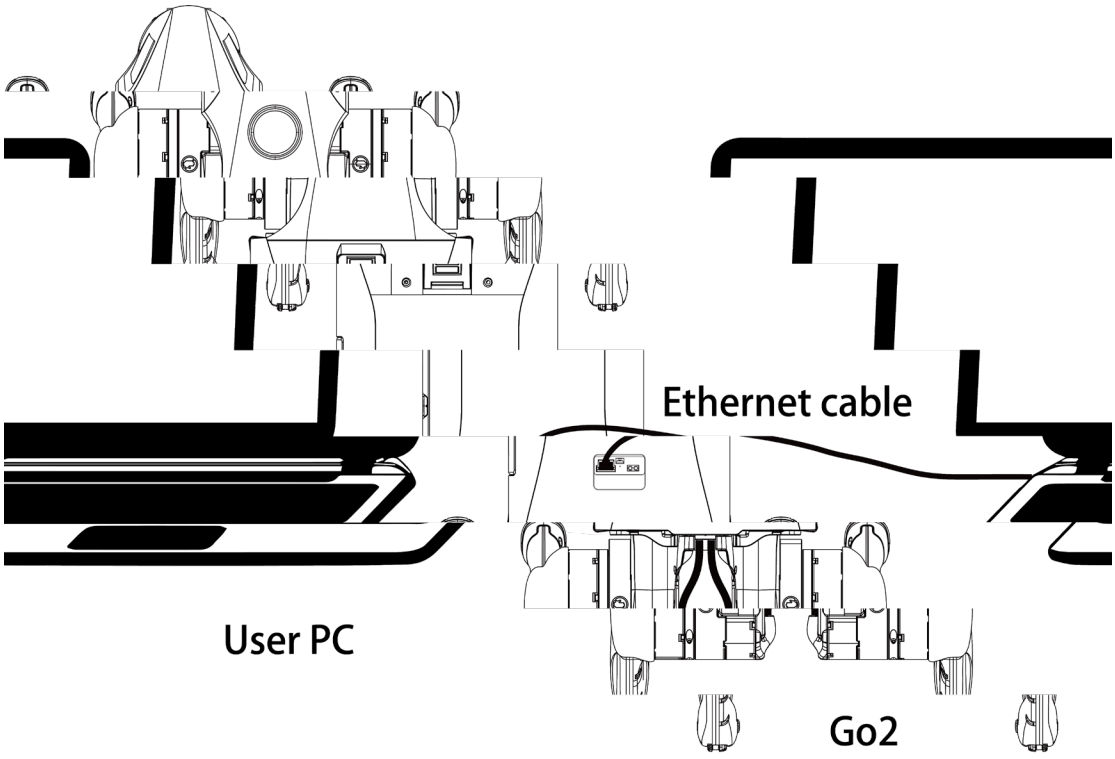


图 3-4 Go2 与用户电脑连接

基于 Go2 机器人的网络连接架构，我们需要将 PC 的 IP 地址设置为 GO2 机器人相关的网段地址。

在终端中使用 **ifconfig** 指令查看网络信息，确认机器人连接到的以太网网卡。(例如如图中的 **enp0s3**，以实际为准)。

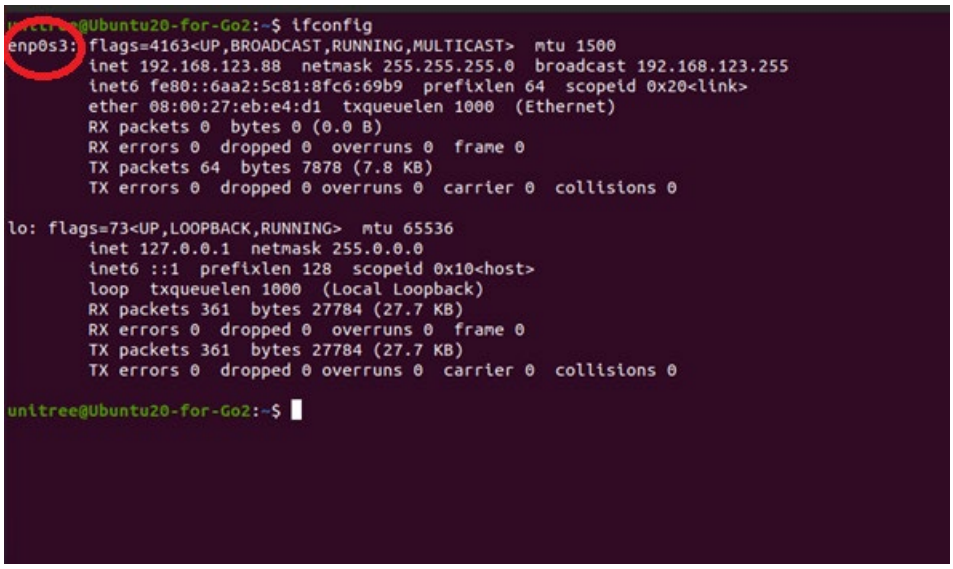


图 3-5 网卡信息确认

接着打开网络设置，找到机器人所连接的网卡，进入 IPv4 ，将 IPv4 方式改为手动，地址设置为 192.168.123.88，子网掩码设置为 255.255.255.0，完成后点击应用，等待网络重新连接。

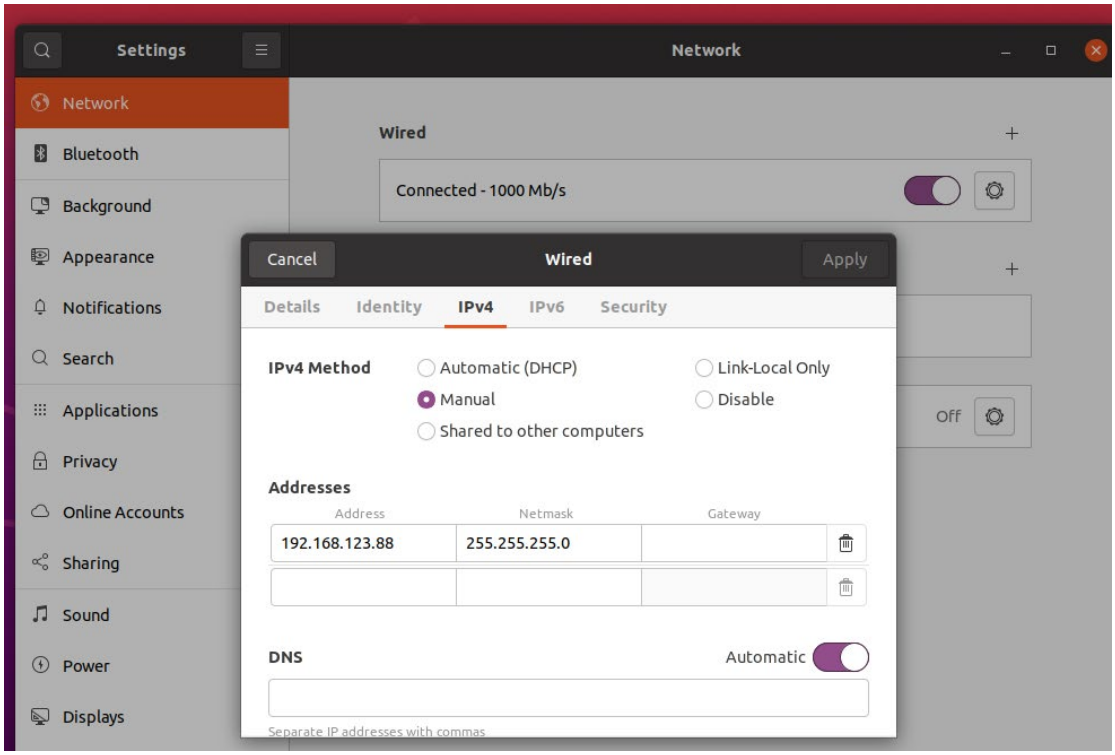


图 3-6 IP 地址修改

Go2 机器人指定的 IP 地址为 **192.168.123.161**，在终端中用 **ping** 指令测试与 GO2 机器人的网络连接情况，连接正常即可进行下一步操作。

```
Ping 192.168.123.161
```

指令 3.4 Ping 连接

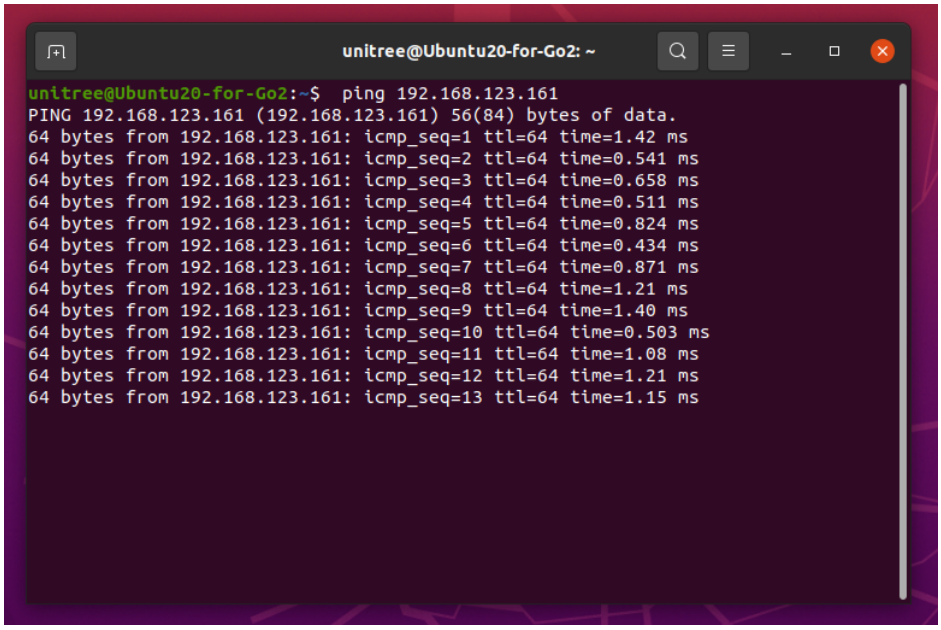


图 3-7 连接测试

3.4、 自定义应用编写及测试

3.4.1、 灯光接口函数

我们将基于灯光控制函数，新建一个控制灯光亮度的应用，应用中用到的接口类和接口函数如下：

类名	构造与析构
VuiClient	explicit VuiClient(); ~VuiClient();
函数名	**SetBrightness**
函数原型	int32_t SetBrightness(int level)
功能概述	设置灯光亮度。
参数	**level**：灯光亮度，取值范围 0~10。
返回值	调用成功返回 0，否则返回相关错误码。

函数名	GetBrightness
函数原型	int32_t GetBrightness(int& level)
功能概述	获取灯光亮度。
参数	**level**：返回的灯光亮度。
返回值	调用成功返回 0，否则返回相关错误码。

表 3-1 灯光控制接口

函数名	GetBrightness
函数原型	int32_t GetBrightness(int& level);
功能概述	获取灯光亮度。
参数	**level**：返回的灯光亮度。
返回值	调用成功返回 0，否则返回相关错误码。
备注	

3.4.2、 例程建立及代码编写

先在 Unitree SDK/example 下新建一个名为 user 的用户目录。

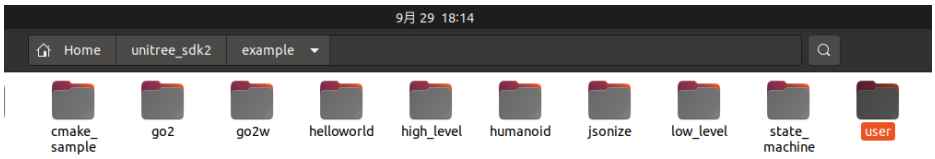


图 3-8 用户应用目录

在 user 目录中创建自己的程序文件 **app_height.cpp**，在文件

中输入以下代码。

核心代码如下：

3.4.2.1、 包含头文件

```
#include <unitree/robot/go2/vui/vui_client.hpp>
```

代码 3.1 灯光控制头文件

3.4.2.2、 初始化 DDS 接口和创建灯光接口对象

```
        * Initilaize ChannelFactory
        */
       std::string networkInterface = argv[1];
       unitree::robot::ChannelFactory::Instance()->Init(0,
networkInterface);
       unitree::robot::go2::VuiClient vc;
```

代码 3.2 初始化接口和对象

3.4.2.3、 灯光亮度控制

```
while (true)
{
    ret = vc.SetBrightness(level);
```

```
        std::cout << "SetBrightness    level=" << level << ", api return:"
<< ret << std::endl;
        ++level %= 11;
        sleep(1);
        ret = vc.GetBrightness(value);
        std::cout << "GetBrightness value=" << value << ", api return:"
<< ret << std::endl;
        sleep(1);
        }
```

代码 3.3 灯光亮度控制

在 user 目录中创建配置文件 **CMakeLists.txt**，在文件中输入以下指令：

```
add_executable(app_height app_height.cpp)
target_link_libraries(app_height unitree_sdk2)
```

指令 3.5 配置文件编辑

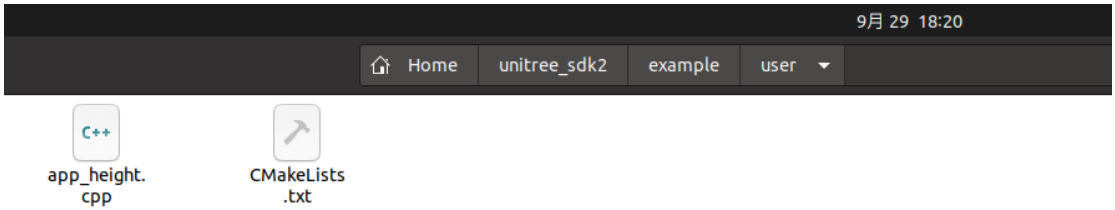


图 3-9 编译配置文件 1

在 user 目录的上级目录中编辑配置文件 **CMakeLists.txt**，在文件中追加以下指令：

```
add_subdirectory(user)
```

指令 3.6 配置文件编辑 2

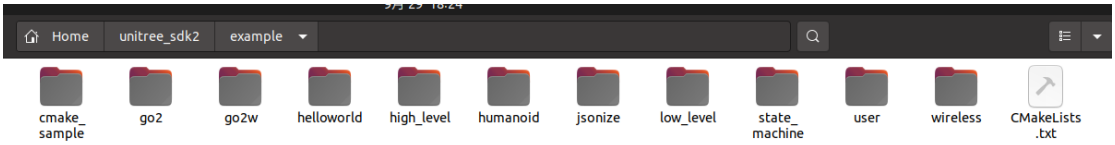


图 3-10 编译配置文件 2

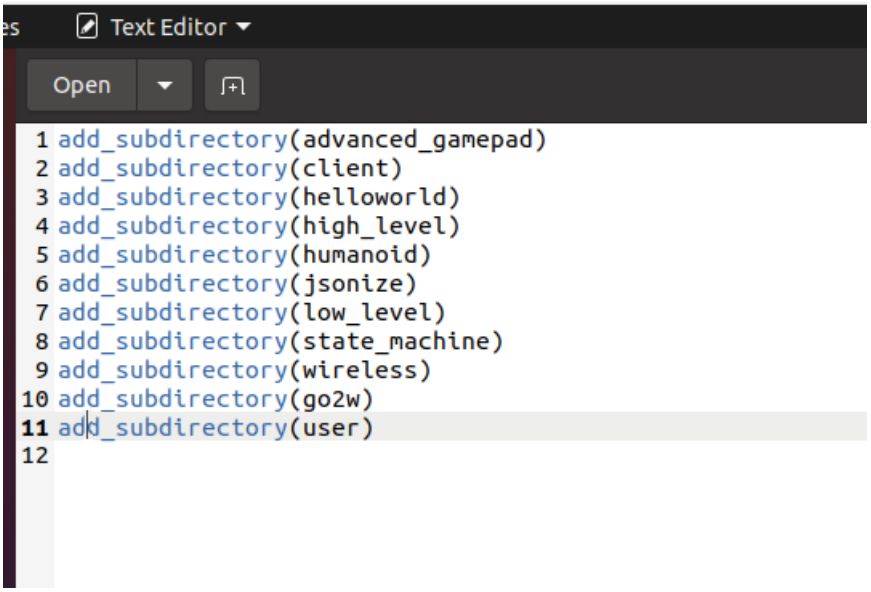


图 3-11 编译配置文件 2-指令追加

3.4.3、 应用编译及应用测试

回到根目录，打开终端，并依次执行下列命令以编译例程：

```
cd ~/unitree_sdk2

cd build

cmake ..

make
```

指令 3.7 例程编译

若成功执行 make 命令, 生成的新例程会在 build/bin 目录下。

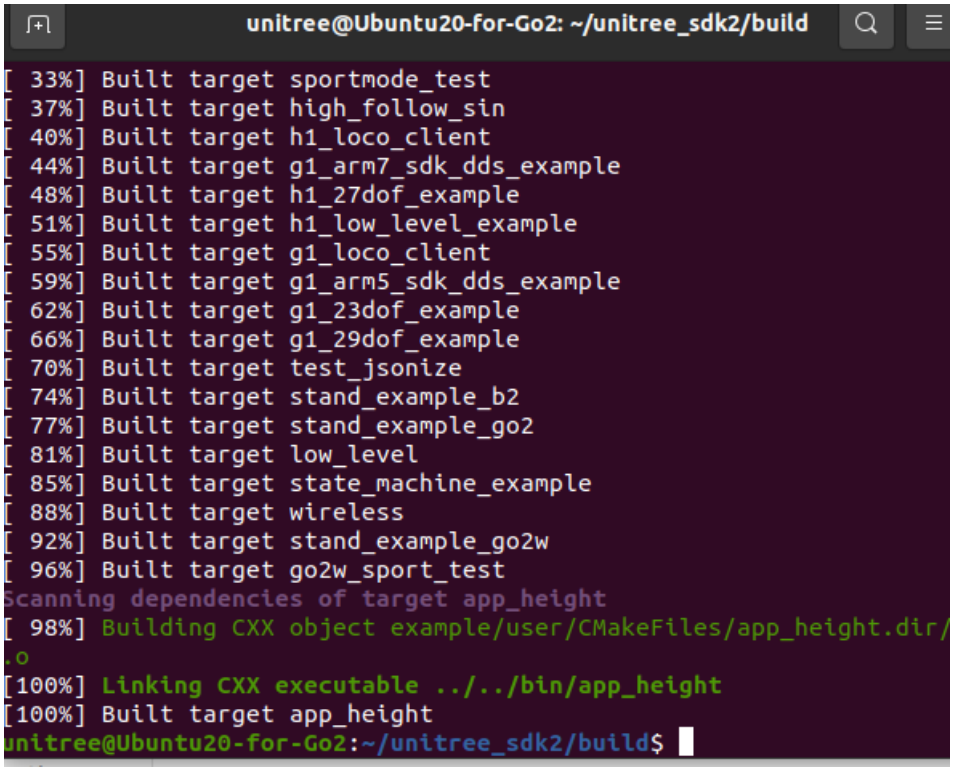


图 3-12 新例程编译

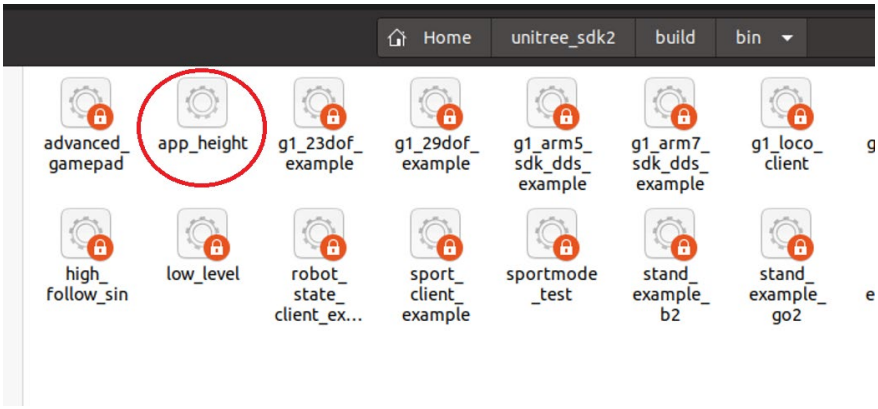


图 3-13 新例程生成

以 Ubuntu 中的超级用户权限，在终端中输入以下指令，测试例程的实际运行结果，此时 Go2 机器人头部彩灯会循环重复从弱到强的亮度变化。

```
cd ~/unitree_sdk2/build/bin

sudo ./app_height enp0s3
```

指令 3.8 例程运行

注意：请根据实际情况替换网卡名 **enp0s3**

4、 Python 开发环境构建

4.1、 Python3 开发环境安装

系统需先具备 Python 开发环境，版本必须在 3.8 以上。

```
sudo apt install -y python3 python3-pip
```

指令 4.1 Python3 开发环境安装

4.2、 unitree_sdk2_python 库安装

4.2.1、 安装流程 1(首选安装方式)

4.2.1.1、 从 Github 仓库上安装 cyclonedds 库

```
cd ~

git clone https://github.com/eclipse-cyclonedds/cyclonedds -b releases/0.10.x
```

```
cd cyclonedds && mkdir build install && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=../install -DBUILD_DDSPERF=OFF
cmake --build . --target install
```

指令 4.2 cyclonedds 库安装指令

4.2.1.2、 Unitree SDK Python 库安装

下载并解压 Unitree_SDK Python 专用库到系统根目录，进入 SDK 文件夹中。

设置 CYCLONEDDS_HOME 为刚刚编译好的 cyclonedds 所在路径，然后执行 unitree_sdk2_python 的安装指令。

```
cd ~
cd ~/unitree_sdk2_python
export CYCLONEDDS_HOME="/{path to your cyclonedds}/cyclonedds/install"
```

```
pip3 install -e .
```

指令 4.3 安装 unitree_sdk2_python

4.2.2、 安装流程 2（次要安装方式，课件 PPT 不体现）

4.2.2.1、安装 cyclonedds 预编译库

在以下网址选择自身开发系统对应的 cyclonedds 预编译库(版本号为 0.10.2)，下载 whl 文件到系统根目录。

https://pypi.org/project/cyclonedds/0.10.2/#files

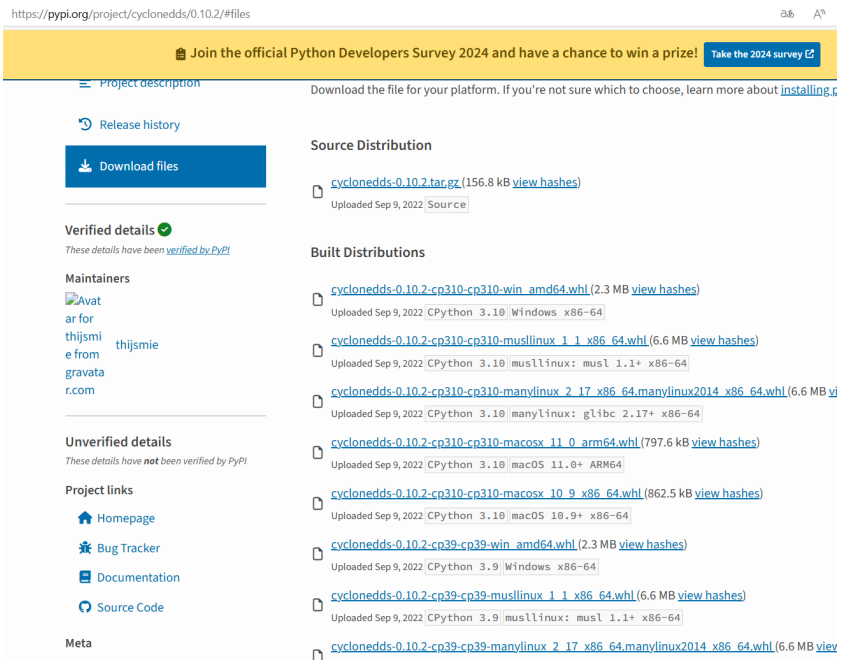


图 4.1 DDS 预编译库选择

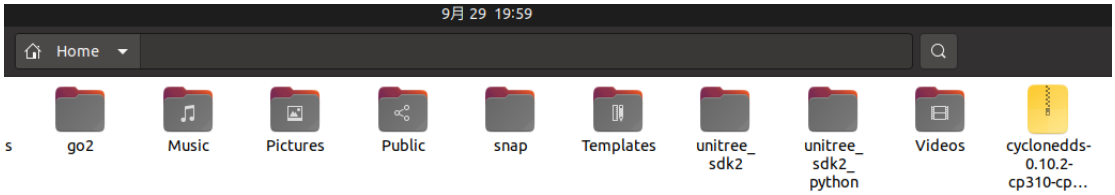


图 4.2 DDS 预编译库下载

```
pip3 install cyclonedds-0.10.2-cp310-cp310-musllinux_1_1_x86_64.whl
```

指令 4.4 DDS 预编译库安装

注意：请根据实际情况替换预编译库名 **cyclonedds-0.10.2-cp310-cp310-musllinux_1_1_x86_64.whl**

4.2.2.2、 Unitree SDK Python 专用库安装

下载并解压 Unitree_SDK Python 专用库到系统根目录，进入 SDK 文件夹中执行安装指令。

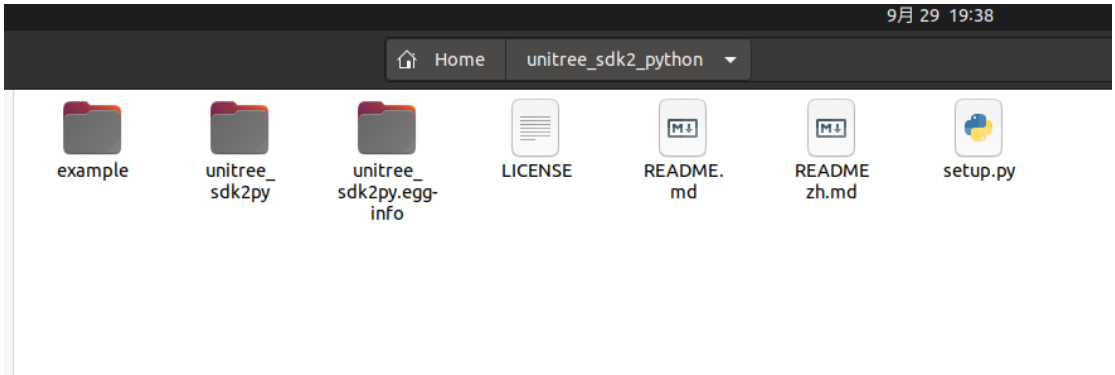


图 4.3 SDK 目录结构

```
cd unitree_sdk2_python

pip3 install -e
```

指令 4.5 Unitree SDK 安装

4.3、 IP 地址设置

详见 **3.3**

4.4、 灯光控制代码编写

在 SDK 的 example/user 目录下新建一个名为 vui_control.py 文件。

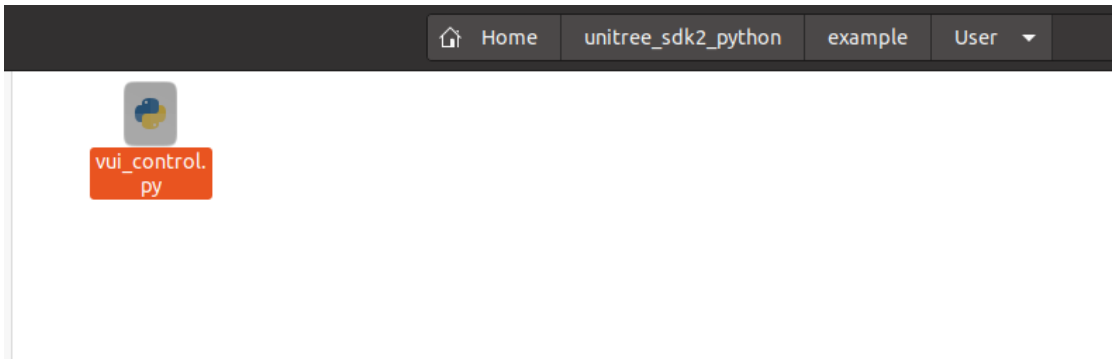


图 4.4 新例程文件

主要代码如下：

4.4.1、 外部包导入

```
import time
import sys
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from unitree_sdk2py.go2.vui.vui_client import VuiClient
```

代码 4.1 包导入

4.4.2、 初始化 DDS 接口和创建灯光接口对象

```
if len(sys.argv)>1:
    ChannelFactoryInitialize(0, sys.argv[1])
else:
    ChannelFactoryInitialize(0)
client = VuiClient()
client.SetTimeout(3.0)
client.Init()
```

代码 4.2 初始化接口和对象

4.4.3、 灯光亮度控制

```
for i in range(1, 11):


print("##################GetBrightness####################
```

```
")
        code, level = client.GetBrightness()
        if code != 0:
            print("get brightness error. code:", code)
        else:
            print("get brightness success. level:", level)
        time.sleep(1)
print("##################SetBrightness####################"
)
        code = client.SetBrightness(i)
        if code != 0:
            print("set brightness error. code:", code)
        else:
            print("set brightness success. level:", i)
            time.sleep(1)
```

代码 4.3 灯光亮度控制

4.5、 灯光控制代码运行及测试

在终端中输入以下指令测试代码，测试例程的实际运行结果，此时 Go2 机器人头部彩灯会循环重复从弱到强的亮度变化。

```
python3  vui_control.py  enp0s3
```

指令 4.6 灯光控制代码控制

注意：请根据实际情况替换网卡名 **enp0s3**

5、 项目总结

通过本项目的学习，我们不仅熟练地搭建了 Go2 机器人的开发环境，还运用了 C++和 Python 两种编程语言，成功编写并调试了一个测试案例，为我们即将开展的二次开发项目奠定了坚实的基础。